Statistical Modelling of Packet-Loss Patterns in Computer Networks: Distribution Fitting, Burstiness Testing and Bayesian Estimation

BAMAT207 Probability and Statistics: Case Study Report

***Abstract*—Loss rate is normally reported as one number, and the easiest models treat every lost packet as if it had nothing to do with the packets around it. Under that view, losses per second are Poisson and the waiting time between losses is exponential. We wanted to know what happens to these results when losses come in bursts instead. To find out, we simulated three networks (wired broadband, campus Wi-Fi and a mobile hotspot) using a two-state Gilbert-Elliott model, with each one sending 50 packets a second for 1 hour, or 180,000 packets in all. On these traces we fitted models for the counts and for the gaps, tested whether neighbouring packets are independent, and estimated loss rates with a Beta-Binomial model. Independence broke down on all three. The variance of the per-second count came out at 1.59, 5.31, 10.17 times the mean (Poisson would give 1), and once a packet was lost, the next one was 31 to 43 times more likely to go too. The practical cost showed up in the error bars. In 2,000 simulated captures, the usual 95% interval for the loss rate held the true value only 40.3% to 85.7% of the time. Cutting the sample size down by a design effect brought that back up to 93.3% to 94.0%. A power study suggests recording at least ten minutes on each network once we start taking real measurements.**

***Index Terms*—packet loss, Gilbert-Elliott model, burstiness, goodness of fit, Bayesian estimation, design effect, Monte Carlo simulation**

I. Introduction

Most of us have run into packet loss without giving it a name. A video call on bad Wi-Fi stalls for a second, a word goes missing from the audio, and then things pick up again. What is happening underneath is simple enough. The app sends small packets at a steady rate and a few of them never make it. The fraction that goes missing is the loss rate. It shows up in service agreements, and it is what protocol designers look at when they tune retransmission timers or pick how much error correction to add.

The catch is that a loss rate says how many packets were lost but not when. Lose ten packets scattered across a minute and the decoder quietly fills each hole; nobody notices. Lose ten in a row and the picture freezes. The standard textbook move is to assume that every packet is lost on its own, with the same small probability p. It is a convenient assumption because it hands you two clean results: the number of losses in a short window is Binomial (so close to Poisson), and the time between losses is exponential. If losses cluster, neither result holds, and any error bars built on top of them are off as well.

So we asked three things. How far does independence actually break? Which of the usual tests notice? And what happens to the interval people quote for the loss rate? The work is planned in two phases, and this report is about the first one, a simulation study. Since we picked every network's true parameters ourselves, we always know the right answer, which lets us check each estimate and each test before we point the same code at real data in Phase 2.

Section II gives some background. Section III describes the loss model and how we generated the data, Section IV goes through the statistical methods, and Section V has the results. We discuss what they mean, and where the study falls short, in Section VI, and wrap up in Section VII.

II. Background

Using a hidden two-state chain for bursty errors is an old idea. E. N. Gilbert proposed it in 1960 for errors on telephone lines, and E. O. Elliott generalised it in 1963 so that either state could produce errors. More than sixty years later, the Gilbert-Elliott model is still where most people start when they want to describe bursty loss.

Nor is it news that Internet losses are not independent. Measurement work in the 1990s found that probe packets sent close together were often lost together, that TCP transfers lost packets in correlated episodes, and that Markov chains did a decent job of describing loss timing in multicast traces. Later studies fitted Gilbert models to voice traffic and tied burst length to how bad a call sounds.

That earlier work mostly set out to describe the loss process. We come at it from the statistics side instead. Which standard tests pick up the dependence, how much data do they need, and how far off are the usual intervals if you ignore it? For that last question we borrowed the design effect from survey sampling. There it is used when people are sampled in clusters, say whole households, rather than one by one, which turns out to be a close match for packets lost in bursts.

III. Loss Model and Data

A. The Gilbert-Elliott Channel

It helps to think of the network as having two moods, Good (G) and Bad (B). We never see the mood itself, only what it does to the packets. Before each packet the channel can switch: Good to Bad with probability p, Bad to Good with probability r. In Good, a packet is lost with a small probability h_G. In Bad, the loss probability h_B is much higher. Over a long run, the channel has to enter Bad as often as it leaves, and that

balance gives the share of time spent in Bad and, from it, the overall loss rate:

$$\pi_B = \frac{p}{p+r}, \qquad P(\text{loss}) = (1-\pi_B)\,h_G + \pi_B\,h_B. \tag{1}$$

How long the channel stays in a state is geometric, with an average of $1/p$ packets in Good and $1/r$ in Bad. A burst is just what happens when the channel drops into Bad for a handful of packets.

B. Scenarios and Sampling

Table I gives the parameters of our three networks. We chose them to cover a low, a middle and a high loss rate, and we let the Bad spells get longer as the network gets worse, since that is roughly what you would expect on a real link. Every network sends 50 packets a second, one every 20 ms, which is about what a voice call does, and we run each for 3,600 s. That works out to 180,000 packets per network and 540,000 overall. All the randomness comes from one seed, 2026, so rerunning the code gives back exactly the numbers in this report.

TABLE I
SCENARIO PARAMETERS AND THE TRUE LOSS RATE THEY IMPLY

Network	p	r	h_G	h_B	π_B	Bad spell	True loss
Wired	0.0005	0.3	0.002	0.5	0.17%	3.3	0.283%
Wi-Fi	0.002	0.15	0.004	0.6	1.32%	6.7	1.184%
Mobile	0.004	0.08	0.006	0.7	4.76%	12.5	3.905%

Bad spell is the average stay in the Bad state, $1/r$ packets. True loss comes from (1).

C. Simulation

The obvious way to simulate the chain is to step it forward one packet at a time. That works, but it is slow. What we do instead is draw the length of each Good or Bad stay straight from the geometric distribution, place the stays one after another, and only then go through the packets and decide which were lost, using the loss probability of whichever state each packet landed in. The process is exactly the same; it just runs much faster. From every trace we take out four things: the loss count in each 1-s window (3,600 per network), the gaps between losses, the burst lengths (runs of back-to-back losses), and how often a delivered (N) or a lost (L) packet is followed by each kind. Fig. 1 plots the first few minutes. You do not need any statistics to see that Wi-Fi and mobile losses come in clumps.

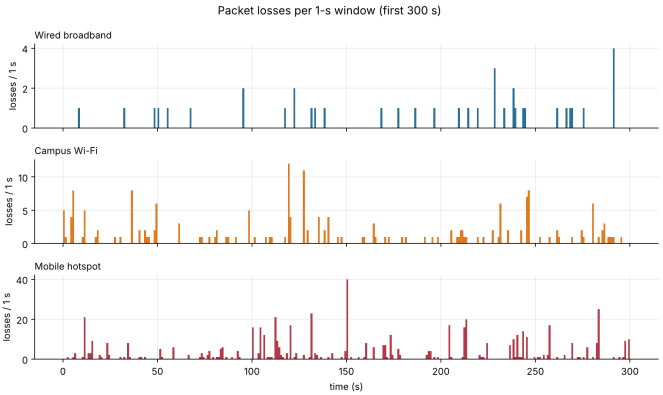


Fig. 1. Losses per second over the first five minutes of each simulated trace. On Wi-Fi and the mobile hotspot, losses arrive in clusters rather than one at a time.

IV. Methodology

A. Fitting the Distributions

For the per-window loss counts $x_1, \ldots, x_n$, the natural first model is a Poisson, and its maximum-likelihood estimate is just the average count, $\hat{\lambda} = \bar{x}$. The natural second model is the Negative Binomial:

$$P(X=k) = \binom{k + r_{NB} - 1}{k} q^{r_{NB}} (1-q)^k, \tag{2}$$

$$\operatorname{Var}(X) = \mu + \mu^2 / r_{NB}. \tag{3}$$

A handy way to think of the Negative Binomial is as a Poisson whose rate does not sit still, which is what you get when a network keeps switching between good and bad stretches. The μ^2/r_{NB} term is what lets the variance climb above the mean, something a Poisson simply cannot do. We fit it by fixing the mean at $\bar{x}$ and searching over r_{NB} for the highest likelihood.

For the gaps between losses, the exponential is the natural first model, with $\hat{\lambda} = 1/\bar{t}$. Our alternative mixes two exponentials together, which is known as a two-phase hyperexponential:

$$f(t) = w\lambda_1 e^{-\lambda_1 t} + (1-w)\lambda_2 e^{-\lambda_2 t}. \tag{4}$$

The thinking behind it is that gaps come in two kinds. Short gaps, with the fast rate λ_1, fall inside a burst. Long gaps, with the slow rate λ_2, are the quiet stretches between bursts. The problem is that no gap comes labelled, so we fit the mixture with the Expectation-Maximisation (EM) algorithm. Each round has two steps: first estimate, for every gap, how likely it is to be a short one; then refit w, λ_1 and λ_2 with those likelihoods as weights. We keep going until the fit stops getting better. Models are compared using the Akaike Information Criterion, $\text{AIC} = 2k - 2\ln L$. It rewards a good fit but charges a penalty for each extra parameter, and the lower score wins.

B. Goodness of Fit

For the count models, we line up how many windows actually had each count against how many the model predicts and use the Pearson statistic $\chi^2 = \sum_i (O_i - E_i)^2 / E_i$. The test

gets shaky when expected counts are tiny, which happens in the tail, so we merge tail bins until every expected count is at least 5. Degrees of freedom are the number of bins, less one, less the number of fitted parameters. We also run the index-of-dispersion test. For Poisson counts, $(n-1)s^2/\bar{x}$ follows χ^2_{n-1}, so a variance well above the mean shows up as a huge statistic.

For the gaps we use the Kolmogorov-Smirnov (KS) distance $D = \sup_t |F_n(t) - F(t)|$, the biggest vertical gap between the data's cumulative curve and the model's. There is a trap here that is easy to miss. The usual KS tables assume the model was fixed before you looked at the data. We estimate the rate from the very data we are testing, which makes the fit look better than it is and the p-value too large. Our fix is a parametric bootstrap: simulate a fake data set from the fitted model, refit it, note its D, and repeat, 2,000 times for the exponential and 500 for the mixture. That shows how big D normally gets when the model really is right.

C. Testing Independence

The Wald-Wolfowitz runs test treats the trace as a string of 0s and 1s (delivered or lost) and counts the runs R, the unbroken stretches of the same outcome. With n_1 losses and n_0 deliveries, independence predicts

$$E[R] = \frac{2n_1n_0}{n} + 1, \quad \mathrm{Var}(R) = \frac{2n_1n_0(2n_1n_0 - n)}{n^2(n-1)}. \quad (5)$$

When losses stick together, the string flips between lost and delivered less often than chance would, so R comes out low.

The Markov-chain test gets at the question more directly. Does what happened to one packet change the odds for the next? Under H_0, no: every packet has the same loss probability. Under H_1, yes: there is one probability after a delivered packet, $a = P(L \mid N)$, and another after a lost one, $b = P(L \mid L)$. Writing n_{xy} for the number of times outcome x is followed by outcome y,

$$\hat{a} = \frac{n_{NL}}{n_{NN} + n_{NL}}, \qquad \hat{b} = \frac{n_{LL}}{n_{LN} + n_{LL}}, \quad (6)$$

$$G^2 = 2(\ln L_1 - \ln L_0). \quad (7)$$

Because H_1 has one parameter more than H_0, Wilks' theorem tells us G^2 is roughly χ^2_1 when H_0 holds. The two estimates are worth keeping for their own sake, too. The ratio b/a says how much a loss raises the odds of another, and $1/(1-b)$ is the average burst length.

To compare the three networks we stuck to rank-based tests. The counts are mostly zeros and heavily skewed, so anything that assumes normality would be on thin ice. Kruskal-Wallis compares all three together; pairwise Mann-Whitney tests compare them two at a time, with a Bonferroni correction so that running three tests does not quietly push up the false-alarm rate. A χ^2 test of homogeneity on the lost and delivered totals checks whether the loss proportion is the same on every network. All tests use $\alpha = 0.05$.

D. Bayesian Estimation

For the loss rate itself we went Bayesian. We start from a flat $\mathrm{Beta}(1,1)$ prior, meaning that before seeing any data we treat every loss rate between 0 and 1 as equally likely. After k losses in n packets the posterior is $\mathrm{Beta}(1+k,\ 1+n-k)$, so in practice we just add the observed losses and deliveries to the prior. We report the posterior mean with a 95% credible interval, and for Campus Wi-Fi we also show how the posterior tightens as traffic comes in. To check that the networks really are ranked by loss rate, we draw 200,000 values from each posterior and count how often Wired $<$ Wi-Fi $<$ Mobile comes out.

The burst parameters get the same treatment. Each has its own posterior, $a \sim \mathrm{Beta}(1+n_{NL}, 1+n_{NN})$ and $b \sim \mathrm{Beta}(1+n_{LL}, 1+n_{LN})$, and from these we get credible intervals for b/a and for the burst length. As a sanity check on the independent model, we also ask it how often a window should hold at least 5 losses, and set that next to how often it really does.

E. Design Effect

Why bursts are bad news for interval estimates becomes clear from the variance of a sum:

$$\mathrm{Var}\Big(\sum_i X_i\Big) = \sum_i \mathrm{Var}(X_i) + 2\sum_{i<j} \mathrm{Cov}(X_i, X_j). \quad (8)$$

If the packets were independent, every covariance would be zero and the variance of the loss count would be $np(1-p)$. Inside a burst, though, neighbouring packets tend to go the same way, so the covariances are positive and the true variance is bigger. The textbook interval leaves this out, so it is too narrow.

To fix it, we cut each trace into blocks of $m = 500$ packets (10 s) and compare how much the block counts really vary with how much they would vary under independence:

$$\mathrm{deff} = \frac{s^2_{\mathrm{block}}}{m\,\hat{p}(1-\hat{p})}, \qquad n_{\mathrm{eff}} = \frac{n}{\mathrm{deff}}, \quad (9)$$

(deff is never allowed below 1.) One way to read n_{eff} is as the number of independent packets that our correlated ones are worth. The adjusted posterior plugs in these effective counts, $\mathrm{Beta}(1+k/\mathrm{deff},\ 1+(n-k)/\mathrm{deff})$, and the interval comes out wider and more honest.

F. Monte Carlo Checks

Some questions cannot be settled from one trace, so we repeat the whole experiment many times. The first is power, or how likely each test is to spot the bursts. For capture lengths from 15 s up to 600 s we simulate 2,000 captures per network and count how often each test rejects independence. The second is coverage, or how often a 95% interval actually contains the true loss rate. Here we simulate 2,000 captures of 10 minutes per network, build both the naive and the adjusted interval for each one, and count the hits. With 2,000 repeats, each proportion is good to within about 0.011 (one standard error).

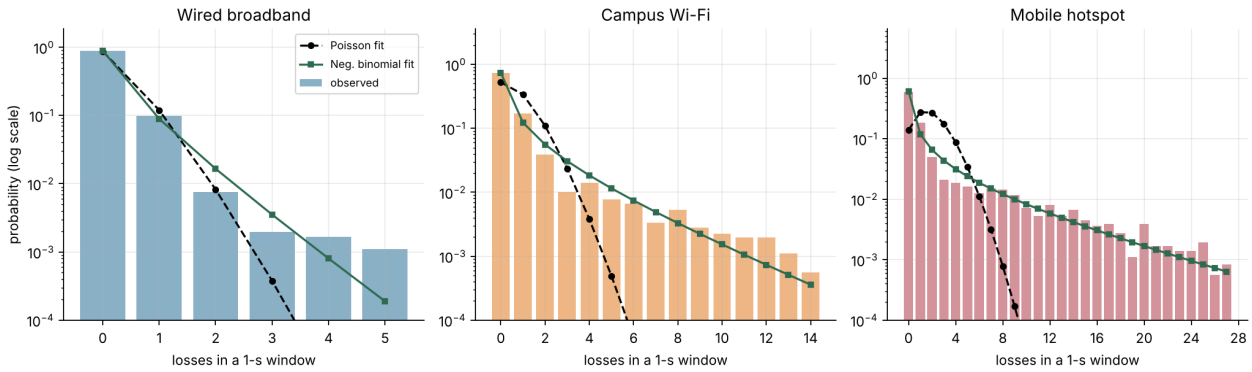


Fig. 2. Probability of k losses in one second, on a log scale. The Poisson fit (dashed) drops far below the real tail; the Negative Binomial (green) follows it.

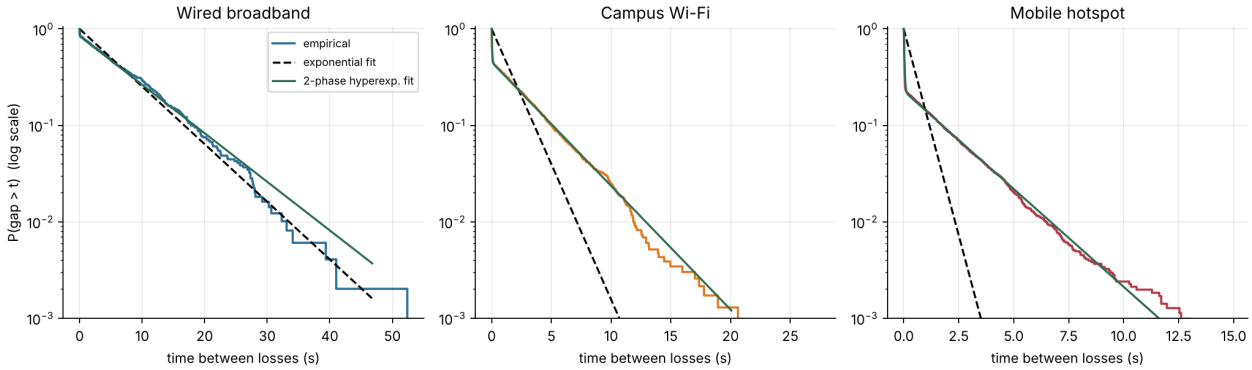


Fig. 3. Probability that a gap between losses is longer than t, on a log scale. An exponential gives a straight line (dashed); the two-phase mixture (green) bends with the data.

TABLE II
LOSSES PER SECOND AND HOW WELL THE COUNT MODELS FIT

Network	Loss rate	Mean / s	Variance	D	χ^2 Poisson	AIC Poisson	AIC Neg. Bin.	Seconds with $\geq$5 losses
Wired	0.275%	0.138	0.219	1.59	22.5	3,143	2,992	7 / 0.00132 / 0.9
Wi-Fi	1.292%	0.646	3.430	5.31	2,544.2	10,607	7,186	131 / 1.98 / 124.2
Mobile	3.937%	1.969	20.017	10.17	15,840.8	25,042	11,700	487 / 180 / 492.0

D is the variance divided by the mean. The last column gives the number of seconds with at least 5 losses: observed / expected under Poisson / expected under Negative Binomial. Every Poisson test gives $p < 10^{-10}$.

V. RESULTS

A. Loss Counts

Poisson does not survive on any of the three networks, and Table II shows why. A Poisson count has its variance equal to its mean. Ours comes out at 1.59, 5.31, 10.17 times the mean, and the dispersion test returns $p < 10^{-10}$ each time. The Negative Binomial does far better and has the lower AIC everywhere. Mobile shows the biggest gap, with the AIC falling from 25,042 to 11,700.

Most of the difference is in the tail. Over an hour of Wi-Fi traffic, Poisson expects roughly 1.98 seconds with 5 or more losses. The trace has 131. The Negative Binomial expects 124.2, which is close. In Fig. 2, on a log scale, the Poisson curve dives almost at once while the real counts and the Negative Binomial keep going.

B. Gaps Between Losses

The exponential does not survive either: the bootstrap KS test rejects it on every network (Table III). Fig. 3 makes the problem easy to see. Plotted on a log scale, an exponential survival curve is a straight line, and the real curves are anything but straight. They fall steeply at first, because a lot of the gaps are tiny, and then level off, because the rest are long.

The two-phase mixture follows that shape well. Its fast phase has an average gap of 29 to 33 ms, only one or two packets, so these are the gaps inside a burst. The slow phase averages between 2.1 and 8.6 s, the quiet time between bursts. The fast phase holds 16% of the gaps on Wired but 77% on Mobile, which fits with the mobile link losing more of its packets in bursts. On Wired the mixture passes the KS test

($p = 0.154$), and it brings the AIC down sharply on all three. It is still rejected on Wi-Fi and Mobile, though, so it is a big step forward rather than the whole picture.

TABLE III
GAPS BETWEEN LOSSES: EXPONENTIAL AND TWO-PHASE FITS

Net.	w	Fast (ms)	Slow (s)	KS p exp.	KS p mix.	AIC exp.	AIC mix.
Wired	16%	32	8.6	< 0.0005	0.154	2,950	2,644
Wi-Fi	55%	33	3.4	< 0.0005	< 0.002	6,682	1,368
Mobile	77%	29	2.1	< 0.0005	< 0.002	4,563	-15,477

w is the share of gaps in the fast phase. Fast and slow are the average gap in each phase.

C. Do Losses Depend on Each Other?

They do, and strongly. Both independence tests reject H_0 with $p < 10^{-10}$ on every network (Table IV). On Wi-Fi we counted 3,005 runs where independence would give about 4,593, a z-score of -146.7. The Markov estimates make it easier to picture. A Wi-Fi packet that comes after a delivered packet is lost 0.85% of the time. One that comes after a lost packet is lost 35.4% of the time.

Across Wired, Wi-Fi and Mobile, the posterior medians of b/a are 43, 42, 31, so a single loss makes the next one 31 to 43 times more likely (Fig. 4). The posterior probability that $b > a$ is 1.000 on every network, which leaves no real doubt about the direction. Bursts are short on average, 1.12, 1.55, 2.25 packets, but they turn up far more often than chance would allow.

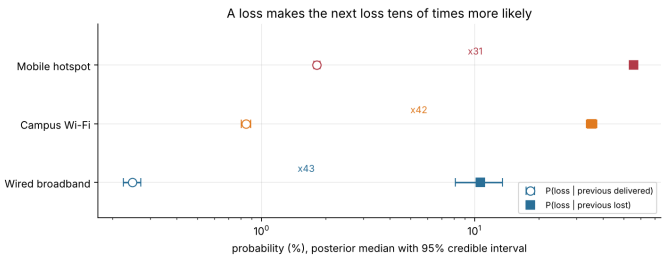


Fig. 4. Posterior medians and 95% credible intervals for $P(L \mid N)$ (open) and $P(L \mid L)$ (filled). On every network a loss makes the next loss far more likely.

TABLE IV
TESTS OF INDEPENDENCE AND BURST PARAMETERS

Net.	Runs	$E[R]$	z	G^2	$P(L\|N)$	$P(L\|L)$	b/a	Burst
Wired	887	988	-43.5	288.4	0.25%	10.5%	43	1.12
Wi-Fi	3,005	4,593	-146.7	4,498.6	0.85%	35.4%	42	1.55
Mobile	6,293	13,617	-228.2	18,560.2	1.82%	55.6%	31	2.25

b/a and burst length (in packets) are posterior medians. All $p < 10^{-10}$.

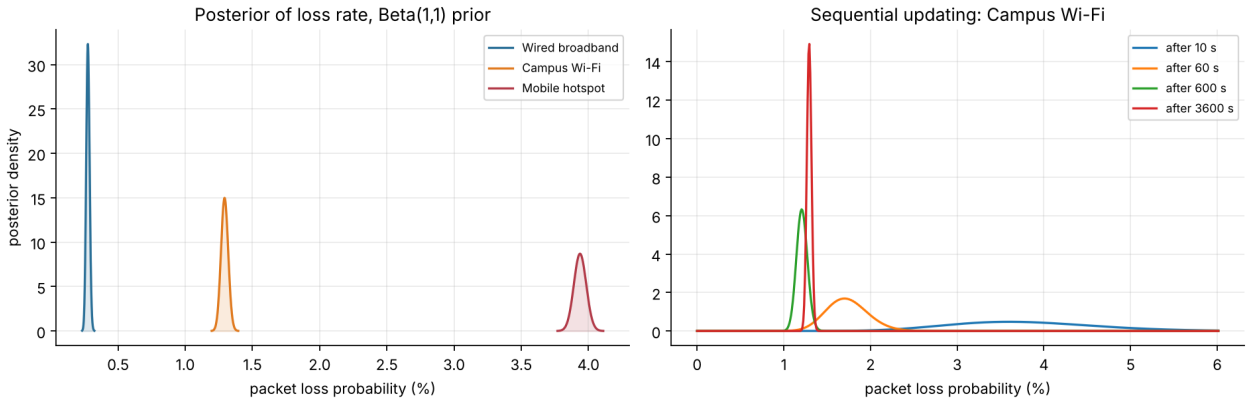


Fig. 5. Left: posterior distributions of the loss rate for the three networks. Right: the Wi-Fi posterior after 10 s, 60 s, 600 s and 3,600 s of traffic.

D. Comparing the Networks

The three networks are clearly different, which is reassuring given that we built them to be. The Kruskal-Wallis statistic is $H = 937.3$, the χ^2 homogeneity statistic is 7,143.0 on 2 degrees of freedom, and even the largest Bonferroni-corrected pairwise p-value is $< 10^{-10}$. The posterior mean loss rates are 0.276%, 1.293%, 3.938% (Table V), and the posterior probability of the order Wired $<$ Wi-Fi $<$ Mobile is 1.000.

Fig. 5 shows the Bayesian estimates. After just 10 s of Campus Wi-Fi traffic the estimate is 3.78% [2.30%, 5.62%], which is wide. By 3,600 s it has narrowed to 1.29% [1.24%, 1.35%].

The predictive check shows how badly the independent model handles heavy seconds. For Wi-Fi it puts the chance of a second with 5 or more losses at 0.00047. In the trace that happens 0.036 of the time, about 77 times as often.

E. How Often Do the Intervals Work?

For us this is the result that matters most. Over 2,000 simulated 10-minute captures, the naive 95% interval held the true loss rate in just 85.7%, 60.9%, 40.3% of cases (Table V, Fig. 6). On the burstiest network it missed more often than it hit.

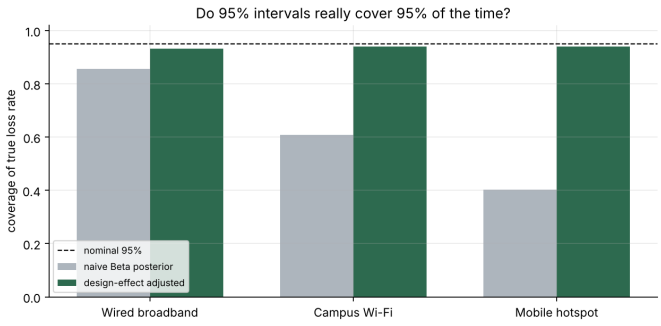


Fig. 6. How often the nominal 95% interval contains the true loss rate, before and after the design-effect correction.

The design effects come out at 1.52, 6.43 and 12.14. For Wi-Fi, that means our 180,000 packets carry about as much information as 28,000 independent ones would. Once we use these smaller effective sample sizes, coverage climbs back to 93.3%, 94.0%, 94.0%, close to the 95% it is supposed to be. The cost is a wider interval: on Wi-Fi the average width goes from 0.245 to 0.563 percentage points. We think that is a fair trade, since a narrow interval that misses the truth is not much use to anyone.

The full Wi-Fi trace makes this concrete. The true loss rate there is 1.184%. The naive interval, [1.241%, 1.345%], misses it entirely; the adjusted one, [1.167%, 1.431%], does not.

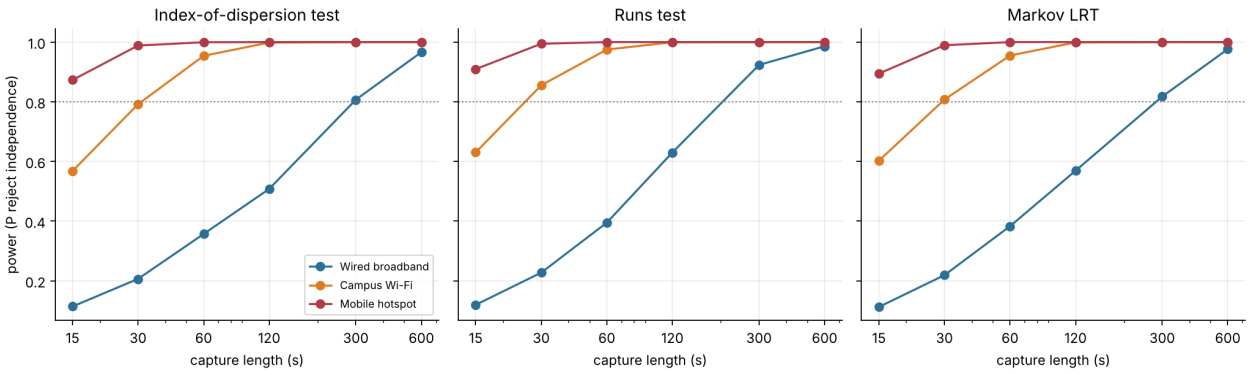


Fig. 7. Power to detect dependence against capture length, from 2,000 simulated captures per point. The dotted line marks a power of 0.8.

TABLE V
LOSS-RATE ESTIMATES AND COVERAGE OF 95% INTERVALS

Network	Posterior mean	Naive 95% interval	deff	Adjusted 95% interval	Naive coverage	Adjusted coverage
Wired	0.276%	[0.252%, 0.300%]	1.52	[0.247%, 0.307%]	85.7%	93.3%
Wi-Fi	1.293%	[1.241%, 1.345%]	6.43	[1.167%, 1.431%]	60.9%	94.0%
Mobile	3.938%	[3.848%, 4.028%]	12.14	[3.636%, 4.263%]	40.3%	94.0%

Intervals are from the full 1-hour trace. Coverage is from 2,000 simulated 10-minute captures.

F. How Long Should a Capture Be?

Fig. 7 shows each test's power rising as captures get longer, and Table VI lists the shortest length we tried that reaches a power of 0.8. With the runs test, Mobile needs only 15 s and Wi-Fi 30 s. Wired needs 300 s: its bursts are rare and short, so evidence builds up slowly. The runs test was the most powerful of the three on every network. At 600 s, every test reaches a power of at least 0.97 everywhere, which is why we plan to record at least ten minutes per network in Phase 2.

TABLE VI
SHORTEST CAPTURE (S) WITH POWER OF AT LEAST 0.8

Network	Dispersion	Runs	Markov	Min. power at 600 s
Wired	300	300	300	0.97
Wi-Fi	60	30	30	1.00
Mobile	15	15	15	1.00

VI. DISCUSSION AND LIMITATIONS

The striking thing is how little it takes. A Bad spell of a few packets is enough to break every model that assumes independence, and the loss rate on its own gives no hint of it. Mobile loses about 14 times as many packets as Wired, yet on both of them one loss makes the next tens of times more likely. Burstiness is its own property of a link and needs to be measured on its own.

The coverage results take this past model choice. A naive interval treats every packet as a new, independent piece of evidence. When packets in a burst share the same fate, there is much less independent evidence than the packet count suggests, and the interval ends up too tight. The burstier the network, the worse this gets, which is why Mobile has the lowest naive coverage. The design-effect fix is cheap (all it needs is block counts) and brings coverage close to 95% on all three networks.

There are clear limits to what we can claim. The biggest one is that the data are simulated, and from the very kind of model that motivated the study. The tests do not use this, and we only use the true parameters to judge the estimates, but real links have effects our model leaves out, busy hours being the obvious one. That is what Phase 2 is for.

A few smaller points. The dispersion test compares the counts with a Poisson, but counts out of 50 packets are strictly Binomial, with a variance a little under the mean. That makes the test slightly conservative, so if anything it understates burstiness. The runs and Markov tests rely on large-sample approximations. In very short captures their false-alarm rate can move away from 5%, though on hour-long traces it stays close. The Negative Binomial and the two-phase mixture are both still rejected on Wi-Fi and Mobile, so neither is the full story. The design effect depends on the block length; ours is 10 s, much longer than any Bad spell, which should be enough. Last, the headline numbers come from a single seeded trace per network, although the Monte Carlo checks repeat the whole experiment thousands of times.

VII. CONCLUSION AND FUTURE WORK

We set out to test whether packet losses on three simulated networks behave independently. They do not. Poisson and exponential models were rejected everywhere. Losses per second were 1.59, 5.31, 10.17 times more spread out than a Poisson allows, the gaps split into short in-burst gaps and long quiet ones, and one loss made the next 31 to 43 times

more likely. The finding we would stress most is about the error bars: the usual 95% interval for the loss rate was right only 40.3% to 85.7% of the time, and dividing the sample size by the design effect lifted that to 93.3% to 94.0%.

Phase 2 comes next. We will record steady streams of 50 packets a second on real wired, Wi-Fi and mobile links, at least ten minutes each, and run the same code on them. After that, there are a few things we would like to try: estimating the Gilbert-Elliott parameters straight from the real traces with the Baum-Welch algorithm, fitting richer models to the tail, using the Phase 1 results as priors in a hierarchical Bayesian model that pools the networks, and comparing loss at peak and off-peak hours.

Reproducibility

Everything in this report comes from one Python pipeline, built on NumPy, SciPy and Matplotlib and run with seed 2026. Running it again reproduces every table and figure.